PRODUCT REQUIREMENTS DOCUMENT

AI-Powered MCP Integration Analyzer & Generator

Local-first, project-aware developer tooling for discovering, designing, generating, and validating Model Context Protocol integrations.

Document control	Details
Version	1.0 - Proposed baseline
Date	26 October 2026
Status	Draft for product and engineering review
Initial scope	TypeScript, Node.js, Next.js; local CLI with optional AI
Audience	Product owner, engineering, architecture, security, QA

This PRD describes a proposed product, not an existing implementation. Timelines, targets, and technology choices are planning assumptions to validate during discovery.

1. Executive summary

The product is a developer tool that analyzes an existing full-stack application and recommends where and how to introduce Model Context Protocol (MCP). It generates reviewable MCP server or client adapters, test scaffolding, security checks, and an integration report. Its core promise is to reduce manual MCP integration effort while respecting existing business logic, permissions, and deployment boundaries.

Primary delivery: a local command-line interface (CLI) and reusable analysis/generation engine. Future delivery options: IDE extension, desktop UI, web console, CI integration, and SDK.

1.1 Problem statement

- Developers must manually inspect routes, services, contracts, authentication, and dependencies before designing MCP integration.
- Naive API-to-MCP conversion can expose unsafe actions or bypass existing authorization.
- Teams lack consistent evidence that generated MCP tools work, validate inputs, and obey security boundaries.

1.2 Product outcomes

- Produce a traceable inventory of integration candidates with confidence and risk scores.
- Generate minimal, reviewable integration changes rather than rewriting the host application.
- Provide reproducible validation reports with explicit pass, fail, skipped, and not-run states.

2. Goals, non-goals, and success metrics

Category	Definition
Goals	Detect project topology; identify safe MCP candidates; generate adapters; validate contracts; report changes.
Non-goals (MVP)	Universal language support; automatic production deployment; autonomous merging; complete business-rule inference; bypassing auth.
Primary user	Backend/full-stack engineer integrating AI assistants with existing systems.
Secondary users	Tech lead, security reviewer, platform engineer, developer-experience team.

2.1 Proposed success criteria

Metric	Initial target / interpretation
Supported reference apps	At least three fixture repositories with repeatable end-to-end flows.
Generation correctness	At least 90% of generated reference adapters compile and pass fixture tests; benchmark target, not guarantee.
Traceability	100% of recommended tools cite source file/function and explain confidence.
Security gate	No write action enabled by default without explicit developer selection and authorization design.
Time to first report	Under five minutes on a representative small repository; validate experimentally.

3. Personas and user journeys

3.1 Primary user journey: expose existing service as MCP tools

Developer installs CLI, points it at a repository, reviews discovered routes/services, chooses approved read-only operations, selects transport and authorization approach, generates an adapter in a separate worktree, runs validation, and reviews a patch and report before merging.

3.2 Secondary journey: add MCP client to an existing AI backend

Developer identifies an existing server-side AI orchestration layer, configures an allowlisted MCP server, generates an MCP client adapter and tool-call policy, verifies credentials and tool invocation, then reviews the integration.

3.3 Example: chat application

A chat application uses Next.js, Node.js, WebSocket messaging, and PostgreSQL. The analyzer finds a read-only conversation summary endpoint and proposes a getConversationSummary MCP tool. It does not automatically expose sendMessage, deleteConversation, or administrative endpoints. If the developer chooses to enable an action tool, the tool requires scoped user authorization, confirmation, auditing, and negative tests.

4. Functional requirements

ID / Priority	Requirement and acceptance criterion
FR-01 / P0	Local repository intake. Accept explicit project root; honor ignore rules; detect monorepos; never execute repository scripts during initial scan.
FR-02 / P0	Stack detection. Identify package manifests, framework versions, service boundaries, routes, and known auth patterns; show evidence and unknowns.
FR-03 / P0	API/service inventory. Discover Next.js route handlers, Express/Fastify routes, OpenAPI specs, TypeScript exports and selected service calls.
FR-04 / P0	Candidate scoring. Classify read/write, side effects, data sensitivity, required scopes, schema quality, and confidence; flag unknown policies.
FR-05 / P0	MCP design report. Recommend server/client/both/none with rationale, tool/resource/prompt candidates, deployment topology and risks.
FR-06 / P0	MCP server generation. Produce typed tool definitions, runtime validation, service adapters, error mapping, logging hooks and configuration.
FR-07 / P0	Validation. Compile generated code; run selected tests and static checks in isolated execution; persist actual command, exit code and logs.
FR-08 / P0	Safe review. Write to separate directory/worktree; show diffs; no overwrite, merge, push, deploy or production mutations without approval.
FR-09 / P1	MCP client integration. Configure allowlisted servers and tool policies in server-side AI runtime; test discovery and invocation.
FR-10 / P1	AI-assisted recommendations. Optional model summarizes architecture and suggests mappings with cited code evidence; user controls data egress.
FR-11 / P1	Machine-readable output. Export JSON report and Markdown/PDF human report; stable finding IDs.
FR-12 / P2	IDE UI, CI mode, plugin marketplace, Python/Java/.NET adapters, multi-repo orchestration.

5. Non-functional requirements

- Security: least privilege, secure secret handling, explicit consent for model uploads, safe defaults, sandboxed validation, dependency pinning, and audit logs.
- Reliability: deterministic scanning when possible, idempotent generation, reproducible fixtures, graceful partial analysis, and actionable errors.
- Performance: incremental scanning and caching; avoid sending whole repositories to an LLM; enforce timeouts and resource limits.
- Privacy: default local processing, configurable redaction, no hidden telemetry, data retention controls, and clear model-provider disclosures.
- Extensibility: versioned language analyzers, framework detectors, MCP generators, and test runners behind stable interfaces.
- Portability: macOS/Linux first; Windows supported through a later compatibility milestone or verified runtime strategy.

6. MCP fundamentals and integration decisions

MCP defines a protocol for exposing tools, resources, and prompts to compatible clients. It does not itself replace application authorization, business services, API gateways, or data access policy. MCP server adapters should reuse existing application service interfaces and never call databases directly when that would bypass established business rules.

Decision	Use when	Pattern
MCP server	External AI hosts need to invoke selected host-application capabilities.	Thin adapter over existing services; typed input/output schemas.
MCP client	The application AI backend needs tools offered by external MCP servers.	Server-side connector with allowlist, scopes and tool policy.
Both	App exposes capabilities and consumes other MCP services.	Separate bounded modules and credentials.
Neither	No clear AI tool-consumption requirement or insufficient security controls.	Recommend no integration; explain reasons.

6.1 Transport and deployment

Support local stdio MCP servers for developer workflows first. For remotely reachable deployments, design an authenticated HTTP transport compatible with the selected MCP specification/SDK version. Transport selection, lifecycle, session behavior, origin checks, and OAuth details must be verified against the chosen protocol version during implementation.

6.2 Candidate classification rules

- Read-only, well-scoped, well-typed operations are the default preferred candidates.
- Write, delete, financial, admin, and bulk-export operations require elevated review and explicit user intent.
- If ownership, tenant isolation, or authorization cannot be established, mark candidate blocked pending clarification.
- Expose only necessary outputs; redact sensitive fields; limit pagination, payload sizes, and tool invocation rates.

7. Reference system architecture

```
Developer -> CLI / future IDE extension -> Orchestrator -> Repository Scanner -> Project Model / Evidence Store -> Candidate & Risk Analyzer -> Integration Planner -> Template/AI Code Generator -> Isolated Worktree -> Validation Runner -> Findings & Diff Reporter -> Developer Approval -> Pull Request / local patch
```

7.1 Component boundaries

Component	Responsibilities	Input -> output
CLI/UI	Commands, auth/consent, review and status.	Arguments -> workflow request
Scanner	AST, manifests, routes, API contracts, imports.	Repository -> normalized project graph
Policy engine	Classify data/side effects/scopes; gate unsafe tools.	Graph -> candidate decisions
Planner	Choose server/client, boundaries, transport and adapters.	Candidates -> integration plan
Generator	Template-based code with optional model assistance.	Plan -> files/patch
Runner	Compile, test, scan under resource restrictions.	Patch -> evidence-backed results
Reporter	Human + machine-readable findings and traceability.	Evidence -> reports

7.2 Data flow and trust boundaries

Repository content is untrusted input, including README instructions, comments, scripts, and prompt-like strings. The scanner parses without running project code. Model suggestions are treated as untrusted proposals. Generation happens in a dedicated worktree. Execution occurs in a restricted sandbox with no production credentials, controlled filesystem mounts, disabled or allowlisted network access, and bounded CPU/memory/time. Only approved changes cross into the user repository.

8. Technical architecture and technology stack

Layer	MVP choice	Reason / alternative
Runtime	Node.js + TypeScript	Shared ecosystem with target apps; Rust/Go possible later.
CLI	Commander or equivalent	Composable commands and CI compatibility.
Parsing	TypeScript Compiler API / ts-morph; Tree-sitter adapters	Structural discovery with source locations.
Project model	JSON schema + graph-like in-memory representation	Deterministic evidence, portable reports.
MCP	Official TypeScript MCP SDK	Avoid custom protocol implementation.
Validation	Zod or JSON Schema, TypeScript compiler	Typed tool input/output validation.
Tests	Vitest; fixture-based integration tests	Fast feedback and reproducibility.
Security	Semgrep, OSV-Scanner, secret scanner	Layered checks; not a security guarantee.
AI (optional)	Provider adapter for selected LLM	Provider-neutral interface; consent and redaction.
Persistence	Local JSON/SQLite cache initially	No cloud dependency for MVP.

9. Design patterns and engineering principles

Pattern	Where used	Why
Ports and Adapters (Hexagonal)	MCP adapter around existing services	Prevents protocol concerns leaking into domain logic.
Strategy	Framework scanners and transports	Swap Next.js/Express analyzers and stdio/HTTP modes.
Factory + Registry	Language analyzers, generators, validators	Pluggable capability discovery.
Pipeline / Chain of Responsibility	Scan -> classify -> plan -> generate -> validate	Observable staged processing and gates.
Command	CLI actions and generated operation invocations	Explicit side effects and repeatable execution.
Policy-as-code	Tool allowlists, risk thresholds, auth requirements	Reviewable and testable security decisions.
Dependency Injection	Service clients, model providers, filesystem adapters	Testability and isolation.
Adapter / Anti-corruption Layer	Wrap legacy REST/service methods as MCP tools	Avoid duplicating business rules.
Human-in-the-loop	Approval of risky tools, writes, merges	Prevents unreviewed changes.

9.1 Key principles

- Evidence before inference: source locations and contracts take precedence over AI speculation.
- Deterministic templates first; use LLMs for explanations and ambiguous mappings, not unrestricted code mutation.
- No implicit privilege escalation; preserve tenant context, identity, scopes, and auditability.
- Generated code remains conventional source code owned by the host application.

10. Proposed monorepo scaffold

```
mcp-integrator/
apps/cli/ # analyze, plan, generate, validate, report
apps/desktop/ # future optional UI
packages/core/ # workflow orchestration and domain models
packages/scanner/ # filesystem + manifests + AST discovery
packages/analyzers/ # Next.js, Express, Fastify adapters
packages/project-model/ # graph, evidence, candidate schema
packages/policy/ # risk scoring, approval rules, allowlists
packages/planner/ # MCP topology and integration plans
```

```
packages/generator/ # code templates, patches, config
packages/mcp-runtime/ # MCP server/client abstractions
packages/ai-provider/ # optional LLM adapter
packages/validation/ # typecheck, test, scans, sandbox runner
packages/reporter/ # JSON, Markdown, PDF output
templates/node-mcp-server/
templates/nextjs-mcp-client/
fixtures/nextjs-chat-app/
fixtures/express-service/
docs/architecture/ # ADRs, threat model, API contracts
.github/workflows/ # lint, test, fixtures, release
```

10.1 Generated host-project scaffold

```
src/integrations/mcp/server.ts # MCP server bootstrap
src/integrations/mcp/tools/ # approved tool definitions
src/integrations/mcp/adapters/ # existing service calls
src/integrations/mcp/auth/ # identity and scope checks
src/integrations/mcp/schemas/ # validated inputs/outputs
tests/mcp/ # contract and auth tests
mcp-integration.config.json # versioned configuration
docs/mcp-integration-plan.md # decisions and limitations
```

Exact paths are configurable to match the host repository conventions. The generator must detect collisions and avoid overwriting existing files without approval.

11. CLI commands and workflow states

Command (proposed)	Behavior
mcp-integrator analyze ./repo	Read-only scan and evidence-backed project inventory.
mcp-integrator plan ./repo	Create candidates, risk findings, and MCP topology proposal.
mcp-integrator generate --plan plan.json	Generate integration files into isolated worktree.
mcp-integrator validate --worktree PATH	Run permitted typecheck/tests/scans; capture evidence.
mcp-integrator report --format json	Export machine-readable findings and summary.

Workflow states: CREATED -> SCANNED -> PLANNED -> AWAITING_APPROVAL -> GENERATED -> VALIDATING -> READY_FOR_REVIEW or BLOCKED. Any failure creates an explicit failed step with logs and retry guidance. Approval and validation are separate gates.

12. Data contracts and example outputs

12.1 Project inventory record

```
{ projectId, repoRoot, languages, frameworks, services, routes, authSignals, dependencyGraph, evidence[], warnings[] }
```

12.2 Candidate record

```
{ id, name, sourceRef, operationType, inputSchema, outputSchema, sensitivity, authRequirements, confidence, risk, decision, rationale }
```

12.3 Integration plan record

```
{ version, mode: "server|client|both|none", transport, selectedCandidates[], requiredApprovals[], generatedPaths[], assumptions[], unresolvedQuestions[], validationPlan[] }
```

12.4 Validation record

```
{ checkId, command, status: "passed|failed|skipped|not_run", exitCode, startedAt, durationMs, logPath, findings[], toolVersion }
```

Reports must never turn skipped or not-run checks into passed checks. All candidate decisions must be linked to evidence and a reviewer can override recommendations with a recorded reason.

13. Security, privacy, and threat model

Threat	Required control
Prompt injection in repository	Treat source text as data, not instructions; constrain AI tool permissions and validate outputs.
Credential leakage	Do not ingest .env secrets; redact logs; use secret scanning and secure OS credential stores.
Unsafe code execution	Separate sandbox, read-only initial scan, limits on process/network/filesystem.
Authorization bypass	MCP adapters call existing domain services with validated user/tenant context and scopes.
Excessive tool permissions	Allowlist tools; default read-only; approval for write/high-risk operations.
Data exfiltration via LLM	Local-first processing; explicit opt-in for remote inference; configurable redaction.
Supply-chain risk	Pin SDK versions, dependency audit, signed releases where feasible.
SSRF / remote MCP risks	Allowlist remote endpoints, validate redirects/URLs, constrain egress and tokens.
Audit gaps	Record generated diffs, approvals, tool calls, scan versions and test evidence.

13.1 Security acceptance gates

- No secrets appear in generated report fixtures or default logs.
- Cross-tenant requests and unauthorized tool invocations fail in negative tests.
- Tool schema validation rejects malformed input and unexpected fields as configured.
- Write-capable tools remain disabled until an explicit approved configuration enables them.

- Production credentials are never made available to generated code validation workers.

14. Testing and quality assurance

Test level	What to verify
Unit	Scanner classification, risk scoring, generator templates, schema mapping.
Contract	MCP tool discovery, input/output schemas, structured errors, capability negotiation.
Integration	Generated adapter invokes mock existing service; propagates identity and tenant correctly.
Negative security	Unauthorized calls, malformed payloads, secret handling, prompt injection, network isolation.
Regression	Snapshot/golden fixtures for generated patches and reports across SDK updates.
End-to-end	Analyze -> plan -> approve -> generate -> validate -> review on reference apps.
Compatibility	Supported Node/Next.js versions, stdio behavior, supported operating systems.

14.1 Example quality report

Check	Sample status	Evidence
TypeScript compile	PASSED	Compiler exit code 0
MCP tool contract	PASSED	Fixture test log
Tenant isolation	FAILED	Unauthorized fixture unexpectedly returned data
Dependency audit	SKIPPED	Scanner unavailable in offline run
Human approval	PENDING	Write operation not authorized

Illustrative statuses only. The tool must calculate these from actual executed checks, never fabricate results.

15. UX and interface design

15.1 CLI-first experience

Progressive disclosure: concise scan summary first, candidate detail second, explicit approval third, then code diff and test report. Show evidence links and risk labels alongside recommendations. Never bury a blocked security finding under a generic success message.

15.2 Future desktop / IDE screens

- Project connection and privacy consent screen.
- Architecture overview with frontend/backend/service boundaries.
- MCP candidate catalog with risk, confidence, source references and enable/disable controls.
- Integration plan editor: server/client choice, transport, authentication and destination paths.
- Diff viewer, validation results, approval workflow and exportable report.

15.3 Accessibility

Provide keyboard-first workflows, readable contrast, plain-text alternatives to diagrams, screen-reader labels, and non-color status labels.

16. Phased delivery plan

Phase	Deliverables	Exit criteria
0: Discovery (1-2 weeks)	Reference repositories, protocol/SDK version decision, threat model, benchmark harness.	Approved scope and architecture decision records.
1: Analysis (2-3 weeks)	CLI, TypeScript/Next.js scanner, inventory schema, candidate scoring, reports.	Fixture inventory matches known reference routes.
2: Generation (2-3 weeks)	MCP server templates, worktree patching, schemas, adapters.	Generated fixture server compiles and lists tools.
3: Validation (2-3 weeks)	Sandbox runner, contract tests, auth negative tests, scans, reporting.	Repeatable E2E tests and evidence-backed status.
4: Expansion (later)	MCP client generation, remote transport, IDE integration, more frameworks.	Separate acceptance gates per integration type.

Indicative timeline: approximately seven to eleven weeks for a focused MVP, with overlap possible. Estimates depend on staffing, target repositories, SDK maturity, and security requirements; refine after discovery.

17. Backlog and prioritization

Priority	Stories
P0	Read-only repository analysis; source evidence; candidate risk scoring; plan approval; stdio MCP server generation; isolated worktree; compile/test/report.
P1	MCP client generation; optional AI mapping; incremental analysis; CI export; authenticated HTTP deployment guidance.
P2	IDE extension; desktop UI; Java/Python/.NET support; enterprise policy packs; multi-repository service map.

18. Dependencies, assumptions, and open decisions

18.1 Assumptions

- Initial target projects are local TypeScript repositories using Node.js or Next.js.
- Users have legitimate permission to inspect and modify connected repositories.
- The MVP can run locally without mandatory cloud accounts or source-code uploads.
- Existing business service interfaces can be reused for at least one fixture use case.

18.2 Decisions to resolve during kickoff

- Exact MCP protocol and SDK versions, transport support, and compatibility policy.

- Supported Node.js and Next.js version matrix; monorepo and package-manager support.
- Approval model for tool exposure, especially write operations and sensitive data.
- Sandbox strategy per operating system; offline mode and dependency install policy.
- AI provider options, opt-in data handling, and whether fully local inference is required.
- Output ownership, licensing, and enterprise distribution strategy.

19. Definition of done and release gates

- The tool analyzes three representative fixture repositories and emits traceable inventories.
- It proposes only reviewed candidates and explains excluded/blocked operations.
- It generates a functioning stdio MCP server adapter for at least one fixture without modifying core business logic.
- Generated code compiles, tests run in isolation, and failures appear clearly in reports.
- No unauthorized writes, source uploads, repository pushes, or production deployments occur.
- Security review, dependency review, documentation, CLI help, and reproducible release artifacts are complete.

20. Future product direction

The second idea can become a standalone MCP engineering product and later a module within the broader AI project-aware development platform. Reuse the scanner, project model, policy engine, sandbox, and reporting interfaces across both products. Avoid building a full autonomous coding agent until the focused MCP workflows are validated with real teams.

Appendix A. Example implementation walkthrough

Input: an existing Next.js chat app with an authenticated server-side getConversationSummary service. The scanner detects the service and identifies its input/output types. The planner recommends a read-only MCP tool and notes that user/tenant context must be carried through. The generator creates an adapter that calls the existing service, validates conversationId, maps errors safely, and emits a stdio server configuration. The runner executes typechecking and mocked service tests, including unauthorized access. The developer reviews the patch, findings, and evidence before applying it.

Appendix B. Suggested architecture decision records

- ADR-001: local-first CLI as MVP delivery surface.
- ADR-002: thin MCP adapters over existing business services.
- ADR-003: AST-backed analysis with explicit evidence and uncertainty.
- ADR-004: separate worktree and sandboxed execution for generated changes.
- ADR-005: deterministic generation with optional, consent-based LLM assistance.
- ADR-006: read-only default and explicit review for side-effecting tools.